

Lecture 6: Semantic Analysis Intro, SDD & SDT

中文学习笔记

基于课件 6.Semantic_Analysis_Intro_SDD_SDt.pdf 整理

2026 年 6 月 26 日

目录

阅读导引	4
1 编译阶段与语义分析位置 [p1-4]	4
2 为什么需要语义分析 [p5-8]	4
2.1 上下文决定含义 [p5-6]	4
2.2 语义分析要做什么 [p7]	5
2.3 两类实现方式 [p8]	5
3 语法制导翻译、属性、SDD 与 SDT [p9-17]	5
3.1 语法制导翻译 [p9-11]	5
3.2 语义属性 [p12]	6
3.3 SDD: 语法制导定义 [p13-14]	6
3.4 SDT: 语法制导翻译方案 [p15]	7
3.5 SDD 与 SDT 的区别 [p16-17]	7
4 SDD 属性类型与基本概念 [p18-28]	7
4.1 综合属性与继承属性 [p19-23]	7
4.2 例子: 表达式值与变量类型 [p24-25]	8
4.3 副作用、属性文法和标注分析树 [p26-28]	8
5 依赖图与属性值计算顺序 [p29-33]	9
5.1 依赖图 [p29-30]	9

目录	2
5.2 拓扑序和环 [p31–33]	9
6 S-Attributed 与 L-Attributed Definitions [p34–37]	10
6.1 S-Attributed Definition [p35]	10
6.2 L-Attributed Definition [p36–37]	10
7 SDT 的实现与 S-SDD 的 LR 实现 [p38–47]	11
7.1 两类实现路径 [p39–40]	11
7.2 S-SDD 转 postfix SDT [p41–42]	11
7.3 扩展 LR 分析栈 [p43–47]	11
8 L-SDD 到 SDT、消除左递归与属性传递 [p48–60]	12
8.1 L-SDD 转 SDT 的规则 [p48–50]	12
8.2 消除左递归时保留语义动作 [p51–55]	12
9 L-SDD 在 LL/RDP/LR 中的实现 [p61–73]	13
9.1 递归下降实现 [p61–62]	13
9.2 非递归 LL 预测分析实现 [p63–67]	13
9.3 L-SDD 在 LR 中的 marker 技巧 [p68–73]	13
10 语义分析应用：绑定、作用域、符号表 [p74–93]	14
10.1 Binding 与 Scope [p76–80]	14
10.2 符号表是什么 [p81–82]	15
10.3 符号表数据结构 [p83–87]	15
10.4 处理多作用域 [p88–90]	15
10.5 符号表条目和类型信息 [p91–93]	16
11 类型、类型检查与类型系统 [p94–103]	16
11.1 Type 与 Type Checking [p94]	16
11.2 静态与动态类型检查 [p95–96]	17
11.3 Static Typing 与 Dynamic Typing [p97–99]	18
12 课件总结 [p100–103]	18
13 练习题与解答 [p104–129]	18

阅读导引

本讲从语义分析进入编译前端的最后阶段。前几讲的词法分析和语法分析主要回答“源程序长得合不合法”；本讲开始回答“它的含义是否合理、如何把语义信息带给后续阶段”。核心工具是语法制导定义 (SDD) 和语法制导翻译方案 (SDT)，再进一步讨论如何在 LL/LR 分析器中实现属性计算，以及语义分析的典型应用：绑定、作用域、符号表和类型检查。

本讲主线

语义分析 = 给语法树上的符号补充语义属性，并检查/传播这些属性。SDD 描述“属性怎么算”，SDT 描述“动作什么时候执行”。若要在语法分析过程中完成语义分析，就必须让属性依赖顺序与分析器的执行顺序兼容。

1 编译阶段与语义分析位置 [p1-4]

语义分析 (semantic analysis) 位于编译前端的最后阶段。整体流程为：

字符流 → 词法分析 → token stream → 语法分析 → syntax tree → 语义分析。

语义分析之后，编译器进入中间代码生成、优化和目标代码生成。

课件展示了 AST 上带类型等标注的例子。普通 AST 只表达结构，例如 while、赋值、加法、比较等节点；Annotated AST 或 Decorated AST 会在节点上额外标注类型、符号表入口、地址、作用域等信息。后续中间代码生成通常依赖这些语义信息。

Annotated AST / Decorated AST

带标注的 AST 是在抽象语法树节点上附加语义属性后的树。它不只是告诉我们程序结构，还告诉我们每个标识符绑定到哪个定义、表达式类型是什么、变量存储在哪里等。

2 为什么需要语义分析 [p5-8]

2.1 上下文决定含义 [p5-6]

程序使用大量符号，也就是标识符 (identifier)。单看一个标识符的名字，编译器无法知道它表示哪个变量、函数、字段或类型；必须结合上下文判断。

课件用英文句子类比：“He ate it” 在语法上正确，但只有知道前文是 “Sam bought a pizza” 时，“He” 才能指 Sam，“it” 才能指 pizza；如果前文是 “Sam bought a car”，“it” 就不能被 ate 合理使用。

语义分析类似地做两件事：

- 关联：把标识符使用点和它引用的对象联系起来，例如把变量使用绑定到某个声明。
- 检查：判断使用是否合法，例如变量是否先声明后使用，类型是否支持对应操作。

语义比语法更难描述。语法只描述程序形式是否符合文法；语义描述程序执行时的意义。CFG 无法处理很多上下文关系，例如“某个使用点必须匹配前面某个同名定义”。课件指出，这类 def-use 模式类似 $\{wcw \mid w \in (a|b)^*\}$ ，不是普通 CFG parser 能直接检查的内容。

2.2 语义分析要做什么 [p7]

语义分析是编译器前端最后一次拒绝错误程序的机会。它检查前面阶段无法捕获的性质，包括：

- 变量必须先声明再使用。
- 标识符使用时类型一致。
- 表达式类型正确，例如布尔表达式不能当整数加法操作数。
- 函数调用参数个数和类型匹配。
- 继承层次、作用域中变量数量等后续阶段需要的信息。

2.3 两类实现方式 [p8]

方式	思路	特点
属性文法 / Attribute grammar	在语法分析过程中，根据产生式规则计算属性并做检查	可一遍完成，适合和 parser 紧密结合
AST walk	第一遍构造 parse tree 或 AST，第二遍遍历树做语义检查	阶段分离清晰，属性计算顺序更自由

如果先构造树再遍历，只要属性依赖没有环，理论上可以选择合适的遍历顺序；如果想在 parsing 过程中完成，就必须受 LL 或 LR 分析顺序限制。

3 语法制导翻译、属性、SDD 与 SDT [p9-17]

3.1 语法制导翻译 [p9-11]

Syntax Directed Translation

语法制导翻译 (syntax directed translation) 是以程序语法结构为驱动进行语义分析和翻译的方法。解

析过程和 parse tree/AST 用来指导属性计算、语义检查和代码生成。

它的基本做法是增强原来的 CFG：

- 给每个文法符号附加语义属性 (semantic attributes)。
- 给每条产生式附加语义规则 (semantic rules) 或语义动作 (semantic actions)。

这就是 CFG-driven translation：语法结构决定在哪里计算哪些语义信息。

3.2 语义属性 [p12]

Attribute

属性是附着在文法符号上的语义信息，记作 $X.a$ ，其中 X 是文法符号， a 是属性名。属性值可以是数字、字符串、类型、内存位置、寄存器、AST 指针、代码片段等。

常见用途：

- 计算表达式值：属性可保存数值。
- 构造 AST：属性可保存指向 AST 节点的指针。
- 生成代码：属性可保存代码片段或临时变量名。
- 类型检查：属性可保存表达式或标识符的类型。

3.3 SDD：语法制导定义 [p13-14]

Syntax Directed Definition

SDD 是一个 CFG 加上文法符号的属性和产生式的语义规则。它是 CFG 的推广：CFG 只描述语法结构，SDD 还描述每个结构上的语义属性如何计算。

例子：

$$E \rightarrow E_1 + E_2, \quad E.val = E_1.val + E_2.val.$$

这里 val 是属性，语义规则说明父节点 E 的值由两个子表达式的值相加得到。

SDD 只描述依赖关系和计算公式，不要求规则在产生式右部的哪个位置执行。

3.4 SDT：语法制导翻译方案 [p15]

Syntax Directed Translation Scheme

SDT 是在 CFG 产生式右部嵌入程序片段的翻译方案，这些程序片段称为语义动作。语义动作在产生式右部的位置决定其执行时机。

例子：

$$E \rightarrow E_1 + E_2 \{E.val = E_1.val + E_2.val\}.$$

若动作在右部末尾，就表示整条右部都已识别后执行；若动作嵌在中间，就表示识别到该位置时立即执行。

3.5 SDD 与 SDT 的区别 [p16–17]

对比项	SDD	SDT
本质	CFG + 属性 + 语义规则	CFG + 属性 + 嵌入在产生式中的语义动作
定位	高层规则说明	可执行的翻译方案
执行顺序	不直接规定，依赖属性依赖关系	由动作在 RHS 中的位置规定
典型形式	$L.inh = T.type$	$D \rightarrow T \{L.inh = T.type\} L$

语义规则不绑定产生式右部的位置；语义动作必须在具体位置执行。若写成

$$A \rightarrow \alpha \{action_1\} \beta \{action_2\} \gamma,$$

则 $action_1$ 在 α 已产生、 β 尚未处理前执行； $action_2$ 在 $\alpha, action_1, \beta$ 之后、 γ 之前执行。

4 SDD 属性类型与基本概念 [p18–28]

4.1 综合属性与继承属性 [p19–23]

综合属性

综合属性 (synthesized attribute) 由某节点自身和它的子节点属性计算得到，信息通常从子节点向父节点上传。

例子：

$$E \rightarrow E_1 + T, \quad E.val = E_1.val + T.val.$$

父节点 $E.val$ 依赖子节点 $E_1.val$ 和 $T.val$ ，所以 val 是综合属性。

继承属性

继承属性 (inherited attribute) 由某节点的父节点、自身或兄弟节点属性计算得到，信息通常从父节点向子节点传递，或从左兄弟向右兄弟传递。

例子：

$$D \rightarrow TL, \quad L.inh = T.type.$$

声明列表 L 继承类型 $T.type$ ，从而知道后续变量应该登记为 `int` 还是 `float`。

终结符可以有综合属性，例如词法分析器给 $digit.lexval$ 、 $id.lexeme$ 赋值；终结符没有继承属性，因为它们通常是词法分析输入提供的叶子，不再向下展开。

4.2 例子：表达式值与变量类型 [p24-25]

表达式求值 SDD：

$$\begin{aligned} E &\rightarrow E_1 + T & E.val &= E_1.val + T.val, \\ T &\rightarrow T_1 * F & T.val &= T_1.val * F.val, \\ F &\rightarrow digit & F.val &= digit.val. \end{aligned}$$

所有 val 都从叶子往上传，是综合属性。

变量声明 SDD：

$$decl \rightarrow type\ varlist, \quad varlist.dtype = type.dtype.$$

类型信息从 $type$ 传给变量列表，再传给每个 `id`，是继承属性。这样每个变量声明点都能被加入符号表，记录对应类型。

4.3 副作用、属性文法和标注分析树 [p26-28]

Side effect

副作用指属性值计算之外的额外行为，例如生成代码、打印结果、修改符号表。它改变了编译器外部状态，而不仅仅返回一个属性值。

Attribute grammar

属性文法是没有副作用的 SDD。其规则只用其他属性值和常量定义属性值。

Annotated parse tree

标注分析树是在 parse tree 的每个节点上标出属性值的树。它帮助我们观察属性如何沿树传播。

AST 也可以被标注成 decorated AST，例如为每个表达式标注类型，为每个变量标注符号表入口，为每个函数标注返回类型。

5 依赖图与属性值计算顺序 [p29–33]

5.1 依赖图 [p29–30]

Dependency graph

依赖图描述某棵 parse tree 中各属性实例之间的信息流。每个属性实例是图中的一个节点；若 $X.a$ 的计算依赖 $Y.b$ ，则从 $Y.b$ 指向 $X.a$ 建一条有向边。

在计算某个属性前，必须先计算它依赖的所有属性。依赖图的作用就是确定这些属性值是否能被计算，以及以什么顺序计算。

例如声明：

$$\text{float } a, b, c$$

中， $T.type = \text{float}$ 必须先算出，然后传给每层 $L.inh$ ，再用于对每个 id 执行 $addtype(id.entry, L.inh)$ 。

5.2 拓扑序和环 [p31–33]

合法属性求值顺序必须满足：若图中有边 $M \rightarrow N$ ，则 M 对应属性必须先于 N 计算。把有向图嵌入线性序列的过程称为拓扑排序（topological sort）。

核心结论

如果依赖图无环，则至少存在一个拓扑序；如果存在环，则没有拓扑序，也就没有办法在这棵 parse

tree 上完成属性计算。

课件给出的循环例子：

$$A \rightarrow B, \quad A.s = B.i, \quad B.i = A.s + 1.$$

这里 $A.s$ 依赖 $B.i$ ，而 $B.i$ 又依赖 $A.s$ ，形成环，无法求值。

综合属性一般需要先计算子节点再计算父节点；继承属性一般需要先计算父节点和依赖的兄弟节点，再计算当前节点。SDD 同时使用两类属性时，不保证自动存在合法顺序，因此需要 S-SDD 和 L-SDD 这样的受限子类。

6 S-Attributed 与 L-Attributed Definitions [p34-37]

6.1 S-Attributed Definition [p35]

S-SDD

若一个 SDD 中所有属性都是综合属性，则它是 S-attributed definition，简称 S-SDD。

S-SDD 的性质：

- 属性都从子节点向父节点传递。
- 可以按任意自底向上顺序计算属性。
- 很适合在 bottom-up/LR parsing 中实现，因为规约时右部属性已在栈顶，正好可计算左部属性。

表达式求值文法就是典型 S-SDD。

6.2 L-Attributed Definition [p36-37]

L-SDD

L-attributed definition 允许综合属性和受限制的继承属性。对产生式

$$A \rightarrow X_1 X_2 \cdots X_i \cdots X_n,$$

X_i 的继承属性只能依赖： A 的继承属性、 X_i 左侧符号 $X_1, \dots, X_{i-1}$ 的属性，以及 X_i 自身不成环的属性。

L-SDD 的直观含义是：产生式右部中信息只能从左向右传，不能依赖还没处理到的右兄弟。这样 top-down 解析时，走到某个符号前，它需要的继承属性已经可得。

示例：

$$A \rightarrow BC, \quad A.s = B.b, \quad B.i = f(C.c, A.s).$$

它不是 S-SDD，因为出现继承属性 $B.i$ 。它也不是 L-SDD，因为 $B.i$ 依赖右兄弟 $C.c$ 和左部综合属性 $A.s$ ，这些在处理 B 前通常还不可得。

7 SDT 的实现与 S-SDD 的 LR 实现 [p38-47]

7.1 两类实现路径 [p39-40]

SDT 可用两种方式实现：

1. 先构造 parse tree 或 AST，再遍历树执行规则/动作。只要依赖无环，就可安排合适顺序，适用范围广。
2. 在 parsing 过程中执行动作。此时动作顺序受分析器推导/规约顺序限制，只适合特定 SDD/SDT。

课件强调两个重要可实现子类：

- S-SDD，且底层文法可 LR 分析。
- L-SDD，且底层文法可 LL 分析。

7.2 S-SDD 转 postfix SDT [p41-42]

S-SDD 转 SDT 的规则很简单：把每个语义动作放在对应产生式的末尾。所有动作都在 RHS 末尾的 SDT 称为 postfix SDT。

例如：

$$E \rightarrow E_1 + T \quad \{E.val = E_1.val + T.val\}.$$

在 LR 分析中，只有当 $E_1 + T$ 完整出现在栈顶时才会规约为 E ，这时计算 $E.val$ 所需属性都已经在栈中，所以动作可随规约执行。

7.3 扩展 LR 分析栈 [p43-47]

为了在 LR 规约时执行语义动作，栈记录不仅保存状态和文法符号，还要保存属性。对产生式：

$$A \rightarrow XYZ, \quad A.a = f(X.x, Y.y, Z.z),$$

规约前栈顶有 X, Y, Z 及其属性；执行动作后，把这三项替换为 A 和 $A.a$ 。

对应栈操作可写成：

$$stack[top - 2].val = f(stack[top - 2].val, stack[top - 1].val, stack[top].val),$$

$$top = top - 2.$$

表达式 $3 * 5 + 4$ 中, 规约 $T \rightarrow T * F$ 时, 栈顶保存 $T.val = 3$ 、 $*$ 、 $F.val = 5$, 于是计算 $T.val = 15$; 之后再与 $+4$ 规约, 得到最终表达式值 19。

8 L-SDD 到 SDT、消除左递归与属性传递 [p48–60]

8.1 L-SDD 转 SDT 的规则 [p48–50]

L-SDD 转成 SDT 时遵循:

1. 若要计算某个 RHS 非终结符 A 的继承属性, 把动作放在 A 出现之前。
2. 若要计算 LHS 的综合属性, 把动作放在产生式右部末尾。

抽象形式:

$$A \rightarrow B \{C.inh = \dots\} C \{A.syn = \dots\}.$$

先准备 C 的继承属性, 再进入 C ; 等 RHS 完成后, 再计算 A 的综合属性。

例如:

$$T \rightarrow F \{T'.inh = F.val\} T' \{T.val = T'.syn\}.$$

8.2 消除左递归时保留语义动作 [p51–55]

Top-down 实现需要先消除左递归。若原文法:

$$E \rightarrow E + T \mid E - T \mid T,$$

带语义规则:

$$E.val = E_1.val + T.val, \quad E.val = E_1.val - T.val.$$

消左递归后:

$$\begin{aligned} E &\rightarrow T \{R.i = T.val\} R \{E.val = R.s\}, \\ R &\rightarrow +T \{R_1.i = R.i + T.val\} R_1 \{R.s = R_1.s\}, \\ R &\rightarrow -T \{R_1.i = R.i - T.val\} R_1 \{R.s = R_1.s\}, \\ R &\rightarrow \epsilon \{R.s = R.i\}. \end{aligned}$$

这里 $R.i$ 是“已经累计到目前为止的值”, $R.s$ 是“把后续所有 $+T/-T$ 都处理完后的值”。对输入 $9-5+2$, 属性流为:

$$R.i = 9 \rightarrow 4 \rightarrow 6, \quad E.val = 6.$$

一般规则:

$$A \rightarrow A_1 Y \{A.a = g(A_1.a, Y.y)\} \mid X \{A.a = f(X.x)\}$$

可改为:

$$\begin{aligned} A &\rightarrow X \{R.i = f(X.x)\} R \{A.a = R.s\}, \\ R &\rightarrow Y \{R_1.i = g(R.i, Y.y)\} R_1 \{R.s = R_1.s\} \mid \epsilon \{R.s = R.i\}. \end{aligned}$$

9 L-SDD 在 LL/RDP/LR 中的实现 [p61-73]

9.1 递归下降实现 [p61-62]

递归下降 parser 通常为每个非终结符写一个函数。L-SDD 在这种实现中很自然：

- 继承属性作为函数参数传入。
- 综合属性作为函数返回值返回。
- 中间语义动作使用局部变量保存。

例如 T' 的函数可写成接受 $T'.inh$ 参数，返回 $T'.syn$ 。若当前 token 是 $*$ ，就读取 $F.val$ ，计算 $T'_1.inh = T'.inh * F.val$ ，递归调用 T'_1 ，最后返回 $T'_1.syn$ 。若当前 token 是结束符或 Follow 符号，就取 ε 分支并返回 $T'.inh$ 。

9.2 非递归 LL 预测分析实现 [p63-67]

表驱动 LL parser 用栈模拟最左推导。为了执行 L-SDD，需要扩展分析栈：

- Action-record：表示待执行动作。
- Synthesize-record：保存非终结符的综合属性。
- 非终结符的继承属性通常放在该符号对应的栈记录中。
- 非终结符的综合属性可放在紧邻该符号的单独记录中。

难点是：当栈顶非终结符 A 展开为 BC 时， A 的记录会被弹出；如果后面计算 $C.i$ 还需要 $A.i$ 或 B 的属性，就必须提前把这些值复制到后续动作记录中。

因此课件总结两条操作原则：

1. 左部展开时，若后续动作需要左部继承属性，要把它复制给后面的动作记录。
2. 综合属性出栈时，若后续动作需要它，也要复制给后面的动作记录。

9.3 L-SDD 在 LR 中的 marker 技巧 [p68-73]

LR 比 LL 语法分析能力更强；S-SDD 在 LR 中很好实现，因为动作都在产生式末尾。但 L-SDD 的动作可能嵌在 RHS 中间，这不符合 LR 规约时才执行动作的自然机制。

问题例子：

$$A \rightarrow X\alpha \{Y.in = X.s\} Y \mid X\beta \{Y.in = X.s\} Y,$$

若 $|\alpha| \neq |\beta|$ ，则 $X.s$ 相对 Y 的栈位置不同，无法用固定偏移统一取得。

解决方案是引入 marker 非终结符：

$$A \rightarrow X\alpha M_1 Y \mid X\beta M_2 Y,$$

$$M_1 \rightarrow \varepsilon \{M_1.s = stack[top - |\alpha|].s\}, \quad M_2 \rightarrow \varepsilon \{M_2.s = stack[top - |\beta|].s\}.$$

Marker 是只产生 ε 的占位非终结符。它把“中间动作”转换为“某个空产生式规约时执行的末尾动作”，从而重新适配 LR 的规约机制。

抽象改写：

$$A \rightarrow \{B.i = f(A.i)\} BC$$

变为：

$$A \rightarrow MBC, \quad M \rightarrow \varepsilon \{M.i = A.i; M.s = f(M.i)\}.$$

10 语义分析应用：绑定、作用域、符号表 [p74-93]

10.1 Binding 与 Scope [p76-80]

Binding

绑定是把标识符使用点匹配到其定义的过程。定义通常把标识符与内存位置、类型、函数体等信息关联起来。

若程序中有多个同名定义，编译器必须判断使用点绑定到哪一个。例子中外层有 `char x`，内层块又有 `int x`，在不同位置出现的 `x = x + 1` 可能对应不同变量。

Scope

作用域是某个定义可用于绑定的程序区域。在该区域内，标识符使用会绑定到该定义；若使用点不在任何定义作用域内，则产生未定义错误。

两类作用域：

类型	决定方式	特点
静态作用域 / lexical scoping	由程序文本结构决定，绑定到最近的外层定义	编译期可分析，错误少，代码更高效；C/C++、Java、Python、JavaScript 常见
动态作用域	由运行时执行路径决定，绑定到当前执行中最近定义	运行时才清楚，人和编译器都较难判断；Bash、早期 Lisp 等出现过

多数语言后来倾向静态作用域，因为绑定可从代码结构直接看出，编译器也可提前优化。

10.2 符号表是什么 [p81-82]

Symbol table

符号表是记录程序中所有符号信息的数据结构。每个条目代表一个标识符定义，维护当前程序点可达的定义，用于把标识符使用绑定到正确定义。

常见操作：

- `enter_scope()`：进入新作用域。
- `exit_scope()`：退出当前作用域。
- `find_symbol(x)`：查找 x 的当前可见定义。
- `add_symbol(x)`：把 x 的定义加入符号表。
- `check_symbol(x)`：检查 x 是否已在当前作用域定义。

符号表通常在生成可执行文件后可丢弃，因为机器码不再使用变量名；但为了调试器显示源码变量名，编译时可保留 debug symbol table，例如 GCC 的 `-g`。

10.3 符号表数据结构 [p83-87]

符号表访问很频繁，会影响编译前端性能。常见结构对比：

结构	空间	插入/删除	查找
数组	几乎无额外空间	$O(n)$	$O(n)$
链表	额外指针空间	$O(1)$	$O(n)$ ，可把最近访问项移到表头优化
平衡二叉树	比数组/链表更多空间	$O(\log n)$	$O(\log n)$ ，最坏可退化
哈希表	表大小 M 可能大于符号数 N	平均 $O(1)$	平均 $O(1)$ ，冲突链过长会退化

多数编译器选择哈希表，因为符号查找非常频繁，平均 $O(1)$ 的访问时间更有价值。哈希冲突通常用链式法解决。

10.4 处理多作用域 [p88-90]

概念上，每个作用域都需要一张独立符号表。活动作用域可用作用域栈管理：

- 进入作用域时创建新符号表，并把指针压入栈。
- 退出作用域时弹出该符号表。
- 查找标识符时从栈顶向下搜索，先找到的就是最近内层定义。

这种结构自然匹配嵌套块结构，例如 $\{C_1\{C_2\{C_3\}\}\{C_4\}\}$ 。栈顶总是最内层作用域。

10.5 符号表条目和类型信息 [p91–93]

符号表条目通常包含：

- name：标识符字符串。
- kind：变量、函数、结构体类型、类类型等。
- attributes：随 kind 不同而变化的属性列表。

不同符号的属性不同：

- 变量：类型、地址。
- 函数：函数原型、函数体地址。
- 结构体类型：字段名、字段类型、字段大小。
- 类类型：类作用域的符号表。

类型信息可很复杂。例如数组可能有多维大小，结构体可能包含多个字段。因此常把类型信息放在属性列表中。

类型信息用于：

- 代码生成：变量赋值是 4 字节拷贝还是 8 字节拷贝；加法是整数加法还是浮点加法。
- 函数调用翻译：参数和返回值类型决定调用约定。
- 代码优化：帮助编译器理解程序语义。
- 类型检查：判断程序类型使用是否合法。

11 类型、类型检查与类型系统 [p94–103]

11.1 Type 与 Type Checking [p94]

Type

类型是一组值以及定义在这些值上的一组操作。例如 `int` 包含整数值并支持整数加减乘除；`char*` 支

持指针相关操作和数组下标访问。

Type checking

类型检查验证程序中操作和操作数类型是否一致。若所有操作都适用于其操作数类型，则程序在类型意义上是一致的。

例子：

- `char *str = "Hello";` 中，`str[2]` 合法，因为 `char*` 可做下标访问；`str/2` 不合法，因为指针不支持除法。
- `int pi = 3;` 中，`pi/2` 合法；`pi=3.14` 类型不一致，除非语言允许隐式类型转换。

11.2 静态与动态类型检查 [p95–96]

静态类型检查

静态类型检查在编译时通过代码分析判断类型一致性。它依赖声明和符号表收集的信息。

例如：

$$\text{int } a, b, c; \quad a = b + c;$$

若 b, c 都是 `int`，编译器可证明 $b + c$ 仍是 `int`，赋给 a 合法。

静态检查不能发现所有错误，例如整数溢出、某些强制类型转换后的实际风险。

动态类型检查

动态类型检查在运行时检查值携带的类型标签。每次操作前先检查 `type tag`，合法才执行，否则触发运行时类型错误。

动态检查可处理编译时无法确定的情况，例如数组越界、运行时 `downcasting` 等，但会带来运行时开销。总体上，能在编译时发现的错误越多越好。

11.3 Static Typing 与 Dynamic Typing [p97–99]

静态/动态 typing 是语言类型系统特征；静态/动态 type checking 是执行检查的方法。

类型系统	特点	优缺点
Static typing	变量有静态类型，只能保存该类型的值；C/C++、Java 常见	程序员约束更多，但 bug 更少、运行更快
Dynamic typing	变量可在运行时保存不同类型值；Python、JavaScript、PHP 常见	代码更灵活更短，适合脚本和原型；但错误可能延迟到运行时，执行开销更大

静态检查不只用于静态类型语言：动态类型语言若编译器能推断类型，也可做静态检查。动态检查也不只用于动态类型语言：静态类型语言中的 downcasting、数组越界等也可能需要运行时检查。

12 课件总结 [p100–103]

本讲知识点可归纳为三组：

- 1. SDD/SDT 基础：属性、综合属性、继承属性、依赖图、拓扑序、S-SDD、L-SDD。
- 2. SDD/SDT 实现：S-SDD 可转 postfix SDT 并在 LR 规约时执行；L-SDD 可在递归下降中用参数/返回值，在 LL 中扩展栈，在 LR 中引入 marker。
- 3. 语义分析应用：绑定、作用域、符号表、类型信息、类型检查、静态/动态类型系统。

推荐阅读对应 Dragon Book 第 5 章 5.1–5.5 节，分别覆盖语法制导翻译介绍和 top-down/bottom-up LR 中翻译方案的实现。

13 练习题与解答 [p104–129]

题 1：无二义文法的最左/最右推导分析树是否一样？

答案：一样。无二义文法中，一个句子只有一棵分析树。最左推导和最右推导只是推导顺序不同，分析树会隐去具体推导顺序。

题 2：如何判断 LL(1) 文法？

对每个非终结符的候选式 $A \rightarrow \alpha \mid \beta$ ，要求：

$$\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset.$$

若 $\beta \Rightarrow^* \varepsilon$ ，还要求：

$$\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \emptyset.$$

也就是说，一个 lookahead 符号不能同时指向两条候选式。

题 3：LL(1) 表行列数由什么决定？

答案：行数由非终结符个数决定；列数由终结符个数加 1 决定，多出的 1 是结束符号\$。

题 4：活前缀

设最右推导：

$$S \Rightarrow aBb \Rightarrow aaAb$$

句柄为 aA 。活前缀是不越过句柄右端的前缀，因此为：

$$a, \quad aa, \quad aaA.$$

题 5：短语、直接短语与句柄

文法：

$$E \rightarrow T \mid E + T, \quad T \rightarrow F \mid T * F, \quad F \rightarrow (E) \mid a \mid b \mid c.$$

句型 $a * b + c$ 的结果：

短语： $a * b + c, a * b, a, b, c.$

直接短语： $a, b, c.$

句柄： $a.$

题 6：构造 LL(1) 分析表

文法：

$$S \rightarrow A, \quad A \rightarrow \varepsilon, \quad A \rightarrow bbA.$$

$$\text{FIRST}(A) = \{b, \varepsilon\}, \quad \text{FOLLOW}(S) = \text{FOLLOW}(A) = \{\$ \}.$$

	b	$\$$
S	$S \rightarrow A$	$S \rightarrow A$
A	$A \rightarrow bbA$	$A \rightarrow \varepsilon$

题 7：文法 $S \rightarrow A, A \rightarrow A + A \mid B ++, B \rightarrow y$ 的 LR 表与 $y++$

课件给出的关键分析表中，状态 5 在输入+ 时有 $r2/s4$ 冲突，因此该文法不是 LR(1)。可通过改写文法、声明结合性/优先级或指定冲突解决策略处理。

对句子 $y++$ ：

状态栈	符号栈	输入	动作
0	\$	$y++\$$	$s1$
0 1	$\$y$	$++\$$	$r4: B \rightarrow y$
0	$\$B$	$++\$$	$\text{GOTO}(0, B) = 3$
0 3	$\$B$	$++\$$	$s6$
0 3 6	$\$B+$	$+\$$	$s7$
0 3 6 7	$\$B++$	$\$$	$r3: A \rightarrow B++$
0	$\$A$	$\$$	$\text{GOTO}(0, A) = 2$
0 2	$\$A$	$\$$	acc

因此 $y++$ 是该文法的一个句子。

题 8: 文法 $S \rightarrow Sb \mid bAa$, $A \rightarrow aSc \mid aSb \mid a$ 的 LR/SLR 分析

增广文法:

$$(0) S' \rightarrow S, \quad (1) S \rightarrow Sb, \quad (2) S \rightarrow bAa, \quad (3) A \rightarrow aSc, \quad (4) A \rightarrow aSb, \quad (5) A \rightarrow a.$$

LR(0) 项集可写为:

状态	项
I_0	$S' \rightarrow \cdot S; S \rightarrow \cdot Sb; S \rightarrow \cdot bAa$
I_1	$S \rightarrow b \cdot Aa; A \rightarrow \cdot aSc; A \rightarrow \cdot aSb; A \rightarrow \cdot a$
I_2	$S' \rightarrow S \cdot; S \rightarrow S \cdot b$
I_3	$S \rightarrow \cdot Sb; S \rightarrow \cdot bAa; A \rightarrow a \cdot Sc; A \rightarrow a \cdot Sb; A \rightarrow a \cdot$
I_4	$S \rightarrow bA \cdot a$
I_5	$S \rightarrow Sb \cdot$
I_6	$S \rightarrow S \cdot b; A \rightarrow aS \cdot c; A \rightarrow aS \cdot b$
I_7	$S \rightarrow bAa \cdot$
I_8	$S \rightarrow Sb \cdot; A \rightarrow aSb \cdot$
I_9	$A \rightarrow aSc \cdot$

该文法不是 LR(0)。原因是 LR(0) 完成项会对所有终结符规约，因而产生冲突：课件标出有移进-规约冲突和归约-归约冲突。按上面的编号， I_3 有 $A \rightarrow a \cdot$ 且还能在 b 上移进； I_8 同时有两个完成项。

该文法是 LR(1)，因为它是 SLR(1)，自然也属于 LR(1)。关键 FOLLOW 集：

$$\text{FOLLOW}(S) = \{b, c, \$\}, \quad \text{FOLLOW}(A) = \{a\}.$$

SLR(1) 表如下：

State	<i>a</i>	<i>b</i>	<i>c</i>	\$	<i>S</i>	<i>A</i>
0		<i>s1</i>			2	
1	<i>s3</i>					4
2		<i>s5</i>		<i>acc</i>		
3	<i>r5</i>	<i>s1</i>			6	
4	<i>s7</i>					
5		<i>r1</i>	<i>r1</i>	<i>r1</i>		
6		<i>s8</i>	<i>s9</i>			
7		<i>r2</i>	<i>r2</i>	<i>r2</i>		
8	<i>r4</i>	<i>r1</i>	<i>r1</i>	<i>r1</i>		
9	<i>r3</i>					

表中没有同一格多动作，因此是 SLR(1)。

题 9：L-属性定义是否每个属性都是继承属性？

答案：错。L-属性定义允许综合属性，也允许满足从左到右依赖限制的继承属性，并不是所有属性都是继承属性。

题 10：RHS 中间有语义动作是否只能用递归下降实现？

答案：错。中间动作确实更适合 top-down 直接执行，但不代表无法用 LR。可以通过 marker 非终结符把嵌入动作转化为空产生式上的 postfix 动作，再用 LR 栈操作实现。

题 11：SDD 输出题(*a*, (*a*))

文法：

$$S' \rightarrow S, \quad S \rightarrow (L) \mid a, \quad L \rightarrow L_1, S \mid S.$$

语义动作：

$$S' \rightarrow S \{ \text{print}(S.\text{num}) \}, \quad S \rightarrow (L) \{ S.\text{num} = L.\text{num} + 1 \}, \quad S \rightarrow a \{ S.\text{num} = 0 \},$$

$$L \rightarrow L_1, S \{ L.\text{num} = L_1.\text{num} + S.\text{num} \}, \quad L \rightarrow S \{ L.\text{num} = S.\text{num} \}.$$

输入(*a*, (*a*)) 中，左侧*a* 的*S.num* = 0，右侧(*a*) 的内部*a* 为 0，因此(*a*) 对应的*S.num* = 1。外层列表的*L.num* = 0 + 1 = 1，外层括号使最终*S.num* = *L.num* + 1 = 2。输出为：

2.

题 12：判断属性规则是否满足 S-SDD / L-SDD

产生式 $A \rightarrow BCD$ ，四个非终结符都有综合属性 *s* 和继承属性 *i*。

规则	S-SDD?	L-SDD?
$A.s = B.s + D.s$	满足, 只有综合属性	满足, 没有继承属性冲突
$A.s = B.i + C.s$	不满足, 使用继承属性 $B.i$	满足, 未违反从左到右限制
$A.s = D.i, B.i = A.s + C.s, C.i = B.s, D.i = B.i + C.i$	不满足, 存在继承属性	不满足, $B.i$ 依赖右兄弟 $C.s$, 且依赖左部综合属性 $A.s$

题 13: 非标准二进制数值系统

文法:

$$S \rightarrow T \mid \varepsilon, \quad T \rightarrow 0T \mid 1T \mid 0 \mid 1.$$

从右至左, 非零二进制位对应的权值符号交替变化。只使用 val 和 tmp 两个属性, 可按课件表填写:

产生式	语义动作
$S \rightarrow T$	$S.val = T.val$
$S \rightarrow \varepsilon$	$S.val = 0$
$T_1 \rightarrow 0T_2$	$T_1.tmp = 2 * T_2.tmp; \quad T_1.val = T_2.val$
$T_1 \rightarrow 1T_2$	$T_1.tmp = -2 * T_2.tmp; \quad T_1.val = T_2.val + T_1.tmp$
$T \rightarrow 0$	$T.tmp = -1; \quad T.val = 0$
$T \rightarrow 1$	$T.tmp = 1; \quad T.val = 1$

直观解释: tmp 表示当前位置对应的带符号权值; 从右向左移动一位时, 权值绝对值乘 2, 符号交替。若当前位是 0, 只传递后续值; 若当前位是 1, 把当前带符号权值加入 val 。

复习建议

复习本讲建议抓住四组对应关系:

- SDD vs SDT: 一个是规则说明, 一个是带执行位置的方案。
- 综合属性 vs 继承属性: 一个向上传, 一个向下/横向传。
- S-SDD vs L-SDD: 一个只允许综合属性, 一个允许受限继承属性。
- 静态 vs 动态: 作用域、类型系统、类型检查都可能有静态/动态之分, 但含义不同。

做题时优先画出属性依赖关系: 若依赖图无环, 再看是否满足 S-SDD 或 L-SDD 的限制; 若要和 parser 同步执行, 再考虑动作应该放在产生式末尾、中间, 还是需要 marker 改写。